

Mini-project 2 in Advanced Machine Learning (02460)

by Oliver (s194591), August (s224178) & Noah (s224207) on 24 March 2026

Part A: Pull-back geodesics

We train a VAE on the first three MNIST classes using a standard VAE Gaussian prior, a Gaussian likelihood $p(x | z)$, and a two-dimensional latent space. The model is trained on a subset of 2048 images with Adam, learning rate 10^{-3} , batch size 32, and 50 epochs. As in the hand-out, small Gaussian noise is added to the inputs during training, which empirically gives a smoother decoder geometry.

To compute pull-back geodesics, we represent each latent curve by T discrete points with fixed endpoints and optimize only the interior points. Let $f(z)$ denote the mean of the Gaussian decoder. We approximate the pull-back energy by a finite-difference discretization,

$$E(c) \approx \sum_{i=0}^{T-2} \frac{\|f(c_{i+1}) - f(c_i)\|_2^2}{\Delta t},$$

and initialize each curve by linear interpolation between two latent posterior means. The interior points are then optimized with Adam to minimize the discretized energy.

Figure 1 shows 25 randomly sampled latent pairs together with the optimized curves. The three classes are clearly separated in the learned latent space. Short intra-class curves remain close to straight lines, while longer curves between clusters bend through low-density regions instead of cutting directly across the embedding. This indicates that the decoder induces a non-Euclidean geometry in latent space. As a small qualitative stability check, we repeated the training three times and recomputed geodesics for the same 10 latent pairs. The mean Euclidean distances were in the range 3.31–3.45, while the mean pull-back energies

were 76.4–86.6, which suggests that the local geometry is reasonably stable but still sensitive to retraining in sparse regions.

figures/geodesics_main.png

Figure 1: Pull-back geodesics between 25 random pairs of latent posterior means for the Gaussian decoder VAE. Black markers denote endpoints and gray curves show the optimized paths.

Part B: Ensemble VAE geometry

Part B extends the decoder to an ensemble setting and compares the stability of Euclidean and geodesic distances across retrainings. The final CoV plot and discussion are added after the ensemble experiments have been completed.

Code snippets

Discretized pull-back curve energy:

```
1 def curve_energy(decoder, curves):
2     batch_size, num_points, latent_dim = curves.shape
3     decoded = decoder(curves.reshape(batch_size * num_points, latent_dim)).mean
4     decoded = decoded.reshape(batch_size, num_points, *decoded.shape[1:])
5     diffs = decoded[:, 1:] - decoded[:, :-1]
6     dt = 1.0 / max(num_points - 1, 1)
7     return diffs.flatten(start_dim=2).pow(2).sum(dim=(1, 2)) / dt
```

Geodesic optimization with fixed endpoints:

```
1 def compute_geodesics(decoder, starts, ends, num_points, num_steps, lr):
2     curves = linear_paths(starts, ends, num_points)
3     interior = nn.Parameter(curves[:, 1:-1].clone())
4     optimizer = torch.optim.Adam([interior], lr=lr)
5
6     for _ in range(num_steps):
7         optimizer.zero_grad()
8         curves = torch.cat([starts[:, None, :], interior, ends[:, None, :]], dim=1)
9         loss = curve_energy(decoder, curves).mean()
10        loss.backward()
11        optimizer.step()
12
13    return curves
```